

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 1
по дисциплине: Алгоритмы и структуры данных
по теме: Встроенные структуры данных(Pascal/C)

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил:

Цель работы: изучение базовых типов данных языка Pascal/C как структур данных (СД).

Задания

1. Для типов данных (см. Варианты заданий в таблицах 1,2) определить:

1.1. Абстрактный уровень представления СД:

1.1.1. Характер организованности и изменчивости.

1.1.2. Набор допустимых операций.

1.2. Физический уровень представления СД:

1.2.1. Схему хранения.

1.2.2. Объем памяти, занимаемый экземпляром СД.

1.2.3. Формат внутреннего представления СД и способ его интерпретации.

1.2.4. Характеристику допустимых значений.

1.2.5. Тип доступа к элементам.

1.3. Логический уровень представления СД.

Способ описания СД и экземпляра СД на языке программирования.

2. Для заданных типов данных определить набор значений, необходимый для изучения физического уровня представления СД.

3. Преобразовать значения в двоичный код.

4. Преобразовать двоичный код в значение.

5. Разработать и отладить программу, выдающую двоичное представление значений, заданных СД.

6. Обработать программой значения, полученные в результате выполнения пункта 3 задания. Сделать выводы.

7. Разработать и отладить программу, определяющую значение переменной по ее двоичному представлению по следующему алгоритму:

1. Ввести двоичный код в переменную S строкового типа.

2. Преобразовать S в вектор В типа «массив байт».

3. Привести В к заданному типу. Вывести значение.

4. Конец.

8. Обработать программой значения, полученные в результате выполнения пункта 4 задания. Сделать выводы.

a) *longint*

Задание №3

Для преобразования значения типа long int в его двоичное представление:

- 1) Определите целое число типа long int, которое вы хотите преобразовать в двоичное представление.
- 2) Разделите это число на 2, записывая остаток от деления и результат деления на каждом шаге.
- 3) Повторяйте этот процесс до тех пор, пока результат деления не станет равным 0. Все остатки, полученные на каждом шаге, составят двоичное представление исходного числа. Начните с записи остатков с последнего.
- 4) Результатом будет последовательность нулей и единиц, представляющая двоичное представление вашего числа.

Код программы:

```
#include <stdio.h>
#include <math.h>

void longIntToBinary(long int n) {
    if (n < 0) {
        n = abs(n);
        printf("-");
    }

    for (int i = sizeof(n) * 8 - 1; i >= 0; --i) {
        int bit = (n >> i) & 1;
        printf("%d", bit);
    }
}

int main() {
    long int num;
    scanf("%ld", &num);

    longIntToBinary(num);

    return 0;
}
```

Задание №4

Для преобразования двоичного числа в десятичное (обычное) число:

- 1) Начните с самого левого (старшего) бита двоичного числа.
- 2) Умножьте текущий бит на соответствующую ему степень числа 2. Степень начинается с нуля для старшего бита и увеличивается на 1 для каждого следующего бита.
- 3) Продолжайте этот процесс для всех битов двоичного числа, двигаясь от старшего к младшему.
- 4) Сложите все результаты умножения для получения десятичного числа.

Код программы:

```
#include <stdio.h>
#include <string.h>
#include <math.h>

long int binaryToValue(long int binNum) {
    long int decimal = 0;
    char binary[64];
    sprintf(binary, "%ld", binNum);
    int binaryLength = strlen(binary);

    for (int i = 0; i < binaryLength; ++i) {
        if (binary[i] == '1') {
            int power = binaryLength - 1 - i;
            decimal += pow(2, power);
        } else if (binary[i] != '0') {
            return -1;
        }
    }

    return decimal;
}

int main() {
    long int num;
    scanf("%ld", &num);

    printf("%ld", binaryToValue(num));
    return 0;
}
```

Задание №5

```
#include <stdio.h>
#include <math.h>

void longIntToBinary(long int n) {
    if (n < 0) {
        n = abs(n);
        printf("-");
    }

    for (int i = sizeof(n) * 8 - 1; i >= 0; --i) {
        int bit = (n >> i) & 1;
        printf("%d", bit);
    }
}

int main() {
    long int num;
    scanf("%ld", &num);

    longIntToBinary(num);

    return 0;
}
```

Задание №6

Тестовые данные

Входные данные	Выходные данные
0	00000000000000000000000000000000
5	00000000000000000000000000000000101
9 223 372 036 854 775 807	11111111111111111111111111111111
-5	-00000000000000000000000000000000101

Вывод : данная программа обрабатывает все значения, которые входят в longint, обрабатывая даже отрицательные значения, но выводит много ведущих нулей

Задание №7

```
#include <stdio.h>
#include <string.h>

long int binaryStringToLongInt(const char* binaryStr) {
    long int result = 0;
    int length = strlen(binaryStr);

    for (int i = 0; i < length; ++i) {
        if (binaryStr[i] == '1') {
            result = (result << 1) | 1;
        } else if (binaryStr[i] == '0') {
            result = result << 1;
        } else {
            printf("Ошибка");
            return -1;
        }
    }

    return result;
}

int main() {
    const char* binaryStr = "101010";
    long int num = binaryStringToLongInt(binaryStr);

    if (num != -1) {
        printf("%ld", num);
    } else {
        printf("Произошла ошибка при преобразовании.\n");
    }

    return 0;
}
```

Задание №8

Тестовые данные

Входные данные	Выходные данные
0	0
110001	49
101010101	341
1111111111	1023

Вывод : данная программа обрабатывает значения от 0 до 1023, отрицательные числа не обрабатывает.

b) comp

Задание №3

Для преобразования комплексного числа, заданного структурой, в двоичный код:

- 1) Определите структуру, которая будет представлять комплексное число. Обычно комплексное число состоит из двух частей: действительной и мнимой.
- 2) Получите значения действительной и мнимой частей комплексного числа из структуры.
- 3) Преобразовать каждую из этих частей в двоичный код. Для этого можно воспользоваться побитовыми операциями и методами, аналогичными тем, которые использовались для преобразования чисел типа double в двоичный код.

После преобразования обеих частей в двоичный код, мы получаем две последовательности нулей и единиц, представляющие действительную и мнимую части.

```
#include <stdio.h>

struct ComplexNumber {
    double real;
    double imag;
} typedef ComplexNumber;

void doubleToBinary(double num, int precision) {
    int integerPart = (int)num;

    int binaryInt[32];
    int index = 0;
    while (integerPart > 0) {
        binaryInt[index] = integerPart % 2;
        integerPart /= 2;
        index++;
    }

    for (int i = index - 1; i >= 0; i--) {
        printf("%d", binaryInt[i]);
    }

    printf(".");

    double fractionalPart = num - (int)num;

    for (int i = 0; i < precision; i++) {
        fractionalPart *= 2;
        int bit = (int)fractionalPart;
        printf("%d", bit);
        fractionalPart -= bit;
    }

    printf("\n");
}

int main() {
    ComplexNumber comp = {206.116, 2.71};

    printf("Binary representation of the real part: ");
    doubleToBinary(comp.real, 10);

    printf("\nBinary representation of the imaginary part: ");
    doubleToBinary(comp.imag, 10);

    return 0;
}
```

Задание №4

Для преобразования двоичного представления в комплексное число:

- 1) Разделите двоичное представление на две части: одна будет представлять действительную часть комплексного числа, а другая - мнимую.
- 2) Преобразовать каждую из этих частей из двоичного представления в десятичное представление. Для этого выполните следующее:
 - а) Начните с самого левого (старшего) бита и двигайтесь к правому (младшему).
 - б) Если бит равен 1, то добавьте соответствующую степень числа 2 к десятичному результату.

3) Повторять этот процесс для каждого бита в двоичном представлении.

```
#include <stdio.h>
#include <string.h>
#include <math.h>

struct ComplexNumber {
    double real;
    double imag;
} typedef ComplexNumber;

double binaryToDouble(const char* binaryValue) {
    double result = 0.0;
    int size = strlen(binaryValue);

    for (int i = size - 1; i >= 0; --i) {
        if (binaryValue[i] == '1') {
            result += pow(2, size - 1 - i);
        }
    }

    return result;
}

int main() {
    const char *binReal = "1001";
    const char *binImag = "1101";

    ComplexNumber comp;
    comp.real = binaryToDouble(binReal);
    comp.imag = binaryToDouble(binImag);

    printf("Complex number: %.2f + %.2fi\n", comp.real, comp.imag);

    return 0;
}
```


Задание №5

```
#include <stdio.h>

struct ComplexNumber {
    double real;
    double imag;
} typedef ComplexNumber;

void doubleToBinary(double num, int precision) {
    int integerPart = (int)num;

    int binaryInt[32];
    int index = 0;
    while (integerPart > 0) {
        binaryInt[index] = integerPart % 2;
        integerPart /= 2;
        index++;
    }

    for (int i = index - 1; i >= 0; i--) {
        printf("%d", binaryInt[i]);
    }

    printf(".");

    double fractionalPart = num - (int)num;

    for (int i = 0; i < precision; i++) {
        fractionalPart *= 2;
        int bit = (int)fractionalPart;
        printf("%d", bit);
        fractionalPart -= bit;
    }

    printf("\n");
}

int main() {
    ComplexNumber comp = {206.116, 2.71};

    printf("Binary representation of the real part: ");
    doubleToBinary(comp.real, 10);

    printf("\nBinary representation of the imaginary part: ");
    doubleToBinary(comp.imag, 10);

    return 0;
}
```

Задание №6

Входные данные	Выходные данные
206.116, 2.71	Binary representation of the real part: 11001110.0001110110 Binary representation of the imaginary part: 10.1011010111
0, 0	Binary representation of the real part: .0000000000 Binary representation of the imaginary part: .0000000000
3, 12	Binary representation of the real part: 11.0000000000 Binary representation of the imaginary part: 1100.0000000000

Вывод : программа обрабатывает любые числа, кроме отрицательных, преобразуя мнимую и действительную часть отдельно.

Задание №7

```
#include <stdio.h>
#include <string.h>
#include <math.h>

struct ComplexNumber {
    double real;
    double imag;
} typedef ComplexNumber;

double binaryToDouble(const char* binaryValue) {
    double result = 0.0;
    int size = strlen(binaryValue);

    for (int i = 0; i < size; ++i) {
        if (binaryValue[i] == '1') {
            result += pow(2, size - 1 - i);
        }
    }

    return result;
}

int main() {
    char binReal[65];
    char binImag[65];

    printf("Enter the binary part for the real part of a complex number: ");
    scanf("%64s", binReal);

    printf("Enter the binary part for the imaginary part of a complex number: ");
    scanf("%64s", binImag);

    ComplexNumber comp;
    comp.real = binaryToDouble(binReal);
    comp.imag = binaryToDouble(binImag);

    printf("Complex number: %.2f + %.2fi\n", comp.real, comp.imag);

    return 0;
}
```

Задание №8

Входные данные	Выходные данные
binReal = "1001"; binImag = "1101";	Complex number: 9.00 + 13.00i
binReal = "0"; binImag = "0";	Complex number: 0.00 + 0.00i
binReal = "11111"; binImag = "11111";	Complex number: 31.00 + 31.00i

Вывод : программа обрабатывает только целые двоичные числа, которые не отрицательные

c) {cat, dog, mouse, tiger}animal

Задание №3

- 1) Задайте уникальные значения для каждого животного:
 - a) cat: 00
 - b) dog: 01
 - c) mouse: 10
 - d) tiger: 11
- 2) Для преобразования значения в двоичный код:
 - a) Определите животное (например, cat).
 - b) Используйте соответствующее ему двоичное представление (00).

```

#include <stdio.h>

enum AnimalType {
    CAT,
    DOG,
    MOUSE,
    TIGER
};

struct Animal {
    enum AnimalType type;
};

void valueToBinary(enum AnimalType animal) {
    printf("Binary value of %d: ", animal);

    int started = 0;

    for (int i = sizeof(animal) * 8 - 1; i >= 0; i--) {
        int bit = (animal >> i) & 1;
        if (bit == 1) {
            started = 1;
        }
        if (started) {
            printf("%d", bit);
        }
    }

    if (!started) {
        printf("0");
    }

    printf("\n");
}

int main() {
    struct Animal animals[] = {
        {CAT},
        {DOG},
        {MOUSE},
        {TIGER}
    };

    printf("Converting values to binary:\n");
    for (int i = 0; i < sizeof(animals) / sizeof(animals[0]); i++) {
        valueToBinary(animals[i].type);
    }

    return 0;
}

```

Задание №4

- 1) Для обратного преобразования из двоичного кода в значение:
 - a) Считайте два бита двоичного кода (например, 01).
 - b) Используйте таблицу соответствия для определения животного (например, dog).

```
#include <stdio.h>

enum AnimalType {
    CAT,
    DOG,
    MOUSE,
    TIGER
};

const char* animalNames[] = {
    "cat",
    "dog",
    "mouse",
    "tiger"
};

struct Animal {
    enum AnimalType type;
    int highBit;
    int lowBit;
};

void binaryToValue(struct Animal* animal) {
    unsigned int binaryValue = (animal->highBit << 1) | animal->lowBit;
    animal->type = (enum AnimalType)binaryValue;
}

int main() {
    printf("\nConverting binary code to values:\n");
    struct Animal animals[] = {
        {CAT, 0, 0},
        {DOG, 0, 1},
        {MOUSE, 1, 0},
        {TIGER, 1, 1}
    };

    for (int i = 0; i < sizeof(animals) / sizeof(animals[0]); i++) {
        binaryToValue(&animals[i]);
        printf("Binary code '%d%d' represents %s\n", animals[i].highBit,
            animals[i].lowBit, animalNames[animals[i].type]);
    }

    return 0;
}
```

Задание №5

```
#include <stdio.h>

enum AnimalType {
    CAT,
    DOG,
    MOUSE,
    TIGER
};

struct Animal {
    enum AnimalType type;
};

void valueToBinary(enum AnimalType animal) {
    printf("Binary value of %d: ", animal);

    int started = 0;

    for (int i = sizeof(animal) * 8 - 1; i >= 0; i--) {
        int bit = (animal >> i) & 1;
        if (bit == 1) {
            started = 1;
        }
        if (started) {
            printf("%d", bit);
        }
    }

    if (!started) {
        printf("0");
    }

    printf("\n");
}

int main() {
    struct Animal animals[] = {
        {CAT},
        {DOG},
        {MOUSE},
        {TIGER}
    };

    printf("Converting values to binary:\n");
    for (int i = 0; i < sizeof(animals) / sizeof(animals[0]); i++) {
        valueToBinary(animals[i].type);
    }

    return 0;
}
```

Задание №6

```
C:\Users\vovad\CLionProjects\vector0nC-
Converting values to binary:
Binary value of 0 : 00
Binary value of 1 : 01
Binary value of 2 : 10
Binary value of 3 : 11
```

Вывод: у нас есть тип `animal`, где каждое животное имеет свой номер, который мы преобразуем в двоичное представление и выводим его

Задание №7

```
#include <stdio.h>
#include <stdlib.h>

enum Animal {
    CAT = 0,
    DOG = 1,
    MOUSE = 2,
    TIGER = 3
};

int main() {
    char binaryCode[9];
    printf("Enter binary code: ");
    scanf("%8s", binaryCode);

    enum Animal animal = (enum Animal)strtol(binaryCode, NULL, 2);

    switch (animal) {
        case CAT:
            printf("Meaning: cat\n");
            break;
        case DOG:
            printf("Meaning: dog\n");
            break;
        case MOUSE:
            printf("Meaning: mouse\n");
            break;
        case TIGER:
            printf("Meaning: tiger\n");
            break;
        default:
            printf("Error: Invalid binary code\n");
            break;
    }

    return 0;
}
```


Задание №8

```
C:\Users\vovad\CLionProjects\vector01  
  
Converting binary code to values:  
Binary code '00' represents cat  
Binary code '01' represents dog  
Binary code '10' represents mouse  
Binary code '11' represents tiger  
  
Process finished with exit code 0
```

Вывод: у нас есть двоичное представление каждого животного, мы преобразуем его в название животного

Вывод : мы изучили базовые типы данных языка *Pascal/C* как структур данных (СД).